

Deterministic Font-Encoding Repair for Nepali–English Code-Switched Retrieval in Legal RAG Pipelines

Pankaj Adhikari

Abstract—Local government documents in Nepal frequently suffer severe semantic degradation when parsed by modern layout extractors, because legacy non-Unicode fonts (e.g., Preeti) surface as corrupted ASCII strings such as ‘‘sf7df8f}F’’ (which encodes *Kathmandu Metropolitan City*). This paper presents an offline-first hybrid extraction and retrieval pipeline for Nepali–English code-switched legal search, and isolates *font-encoding repair* as the decisive preprocessing step. A deterministic document router dispatches text-layer PDFs to PyMuPDF and scanned PDFs to Surya OCR; extracted text is segmented with heading-aware chunking (512-token window, 128-token overlap fallback). Over a curated corpus of 50 municipal and federal legal PDFs yielding 446 retrieval chunks—of which 26 documents are Preeti-corrupted—sparse TF-IDF retrieval on raw corrupted text attains only Recall@3 = 0.060 (MRR = 0.023). We introduce an open, deterministic Preeti→Unicode repair module (glyph substitution plus vowel-sign and reph reordering) that recovers 45.8% token-level validity against an independent clean-document vocabulary and, applied before indexing, lifts sparse retrieval to Recall@3 = 0.520 and document-level Recall@3 = 0.800—an order-of-magnitude gain with *no* hand-authored aliases. On a leakage-free 50-query bank, deterministic repair *outperforms* a hand-authored metadata-aliasing baseline (Recall@3 = 0.260), which helps only the curated chunks it targets. Repair is also necessary for dense retrieval, but it pays off only with a language-appropriate, sufficiently capable encoder: on identical chunk IDs it lifts a smaller multilingual encoder from Recall@3 = 0.040 to 0.160 and leaves an English-centric encoder near-broken (0.060), while a larger multilingual encoder (BGE-M3, CPU-only, peak 1.68 GB) *overtakes* sparse-plus-repair outright, reaching Recall@3 = 0.700 (document-level 0.800). The residual gap to perfect recall is attributable to conjunct lossiness in the deterministic map, motivating learned transliteration repair as future work.

Index Terms—Retrieval-Augmented Generation, Code-Switched NLP, Font Encoding, Preeti, Nepali Legal IR, Document Intelligence, Sparse Retrieval.

I. INTRODUCTION

The digitalization of municipal governance under Nepal’s federal structure has produced a large volume of unstructured, low-resource-language PDFs hosted across Kathmandu, Lalitpur, and Bhaktapur portals, as well as federal statute repositories. These assets remain difficult to query with conventional Large Language Models (LLMs): citizens typically ask in Nepali–English code-switched language (e.g., parking fees, property-tax slabs, business-registration documents), while the underlying PDFs often use legacy non-Unicode fonts. When

extracted by standard text scrapers, glyphs map to randomized ASCII, destroying subword tokenization and inducing embedding-space mismatches for dense retrievers.

Scope. This paper evaluates the *retrieval substrate* of a Retrieval-Augmented Generation (RAG) system—the stage that determines whether a generator can be grounded at all. Generation quality is discussed qualitatively (Section V-A) but is not the object of quantitative claims here.

We address the extraction gap with a privacy-preserving, edge-deployable pipeline that (i) routes documents by curated format type, (ii) chunks with legal heading awareness, (iii) *deterministically repairs Preeti-encoded text to Unicode before indexing*, and (iv) evaluates the effect on sparse retrieval. Our contributions are:

- 1) A reproducible hybrid router (PyMuPDF vs. Surya OCR) and semantic chunker for Nepali municipal PDFs, with a quantified *extraction-yield* audit of the resulting chunks.
- 2) An open, deterministic Preeti→Unicode repair module that reconstructs Devanagari from corrupted ASCII at 45.8% token-level validity, validated against parallel clean-document vocabulary.
- 3) A reproducible 50-query, leakage-free code-switched evaluation showing that deterministic repair alone lifts Recall@3 from 0.060 to 0.520 (document-level Recall@3 from 0.140 to 0.800) with no aliasing, and that it empirically *outperforms* a hand-authored metadata-aliasing baseline (0.260) that does not generalize beyond its curated chunks.
- 4) A cross-retriever ablation on identical chunk IDs—sparse TF-IDF and three dense encoders, each on raw and repaired text—establishing that repair is necessary for dense retrieval but pays off only with a language-appropriate, sufficiently capable encoder, and that under an edge memory budget a capable multilingual encoder (BGE-M3, repaired) overtakes sparse-plus-repair to become the strongest substrate we measure.

II. RELATED WORK AND SYSTEM GAPS

Large language models hallucinate and go stale [14]; retrieval-augmented generation [1] addresses this by grounding generators in retrieved evidence, so retrieval quality bounds end-to-end faithfulness [15]. Dense passage retrieval [2], late-interaction retrievers [16], and multilingual encoders such as BGE-M3 [4] build on Transformer language models [17],

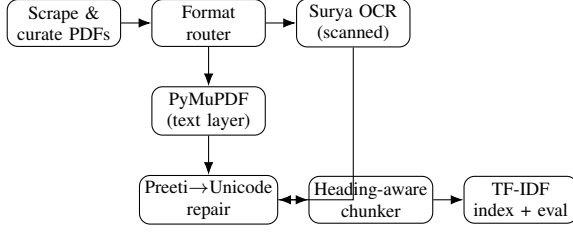


Fig. 1. Offline pipeline. Deterministic encoding repair is inserted between extraction and chunking so that all downstream indexing operates on Unicode.

[18] and cross-lingual pretraining [19], [20]; all assume coherent sub-tokens, so broken glyph mappings eliminate contextual signal and map queries into incorrect vector neighborhoods. Classical sparse methods—TF-IDF and BM25 [3], [7], [21]—remain complementary when text is noisy or out-of-vocabulary, motivating a sparse evaluation path here. Existing document pipelines exhibit recurring failure modes on legacy South Asian corpora:

- 1) **Dense embedding fragility.** Corrupted glyphs produce out-of-vocabulary tokens that carry no learned semantics [4], [5].
- 2) **OCR and digitization bottlenecks.** Devanagari OCR remains hard on low-resolution legal layouts [10]; classical engines such as Tesseract [22] and modern neural pipelines (e.g., Surya [8]) trade coverage for latency.
- 3) **Encoding, not just OCR.** Many Nepali PDFs are *text-layer* documents whose bytes are correct but whose font (Preeti and relatives) lacks a Unicode mapping; the failure is a deterministic glyph substitution, not a recognition error. Prior Nepali NLP work has focused on Unicode-native corpora [9], [23], leaving this legacy-encoding regime under-served; existing Preeti-to-Unicode conversion is limited to undocumented, closed-source web utilities rather than a validated, published methodology.
- 4) **Ingestion integrity.** Extraction errors cascade linearly through chunking and indexing, enforcing a strict “garbage in, garbage out” constraint on generation [13].

Nepali municipal legal IR remains under-studied relative to English common-law corpora, particularly under code-switching [11] and Preeti-style encodings. We position this work as a systems evaluation of preprocessing and retrieval hygiene rather than a new embedding architecture.

III. METHODOLOGY

The pipeline implements five stages: web scraping and curation, hybrid extraction routing, deterministic encoding repair, semantic chunking, and offline retrieval evaluation (Fig. 1).

A. Corpus Construction

From several hundred scraped PDFs we curate **50 documents** spanning federal statutes and Kathmandu Valley municipal bylaws, administrative procedures, and financial schedules [12]. Each record stores `doc_id`, `source`, `category`, `title`, `path`, and `format_type` $\in \{\text{text_based}, \text{scanned}\}$.

TABLE I
DEVANAGARI CHARACTER RATIO: THE QUERIED DOCUMENTS ARE CORRUPTED

Document	Devanagari %	Status
LMC_MUN_004	0.4%	corrupted
BKT_MUN_001	0.0%	corrupted
KTM_FIN_002	0.3%	corrupted
KTM_NAT_001	97.7%	clean
KTM_NAT_004	98.2%	clean

TABLE II
CHUNK-SIZE DISTRIBUTION (CHARACTERS) OVER ALL 446 CHUNKS

Statistic	Characters
Minimum	3
Median	1591
Mean	5178
Maximum	163520

After extraction and chunking the retrieval index contains **446** unique JSONL chunks (not 446 PDFs). A text-layer subset of 10 documents (830 pages) carries per-document extraction telemetry.

Corruption is real and concentrated. Measuring the Devanagari-to-Latin character ratio per document reveals a bimodal corpus: 18 documents extracted as clean Unicode ($\geq 90\%$ Devanagari), while 26 extracted as near-pure Latin gibberish ($< 10\%$ Devanagari). Critically, all three ground-truth target documents of our query bank fall in the corrupted set (Table I), so the retrieval failure we study is not incidental.

Extraction-yield audit. Chunk sizes are strongly non-uniform (Table II): the median chunk is 1591 characters but the maximum is 163520, indicating that heading detection failed on some documents and left them un-split, while a minimum of 3 characters reveals near-empty fragments. We report this rather than silently averaging over it.

B. Document Ingestion and Routing

A deterministic router dispatches each PDF using the curated `format_type` label (not an online visual classifier at inference time):

$$f(d) = \begin{cases} \text{PyMuPDF}, & \text{if } \text{format_type}(d) = \text{text_based}, \\ \text{Surya OCR}, & \text{if } \text{format_type}(d) = \text{scanned}, \\ \text{PyMuPDF (default)}, & \text{otherwise.} \end{cases} \quad (1)$$

Surya OCR [8] runs on Apple Silicon Metal Performance Shaders by default, with optional page-range constraints; per-document duration and page count are logged for confounding-factor analysis. OCR ingestion is a one-time offline corpus-build step and is not part of the reproducible retrieval benchmark; the CPU-only, edge-memory-budget claim in Section III-E and Table IV applies specifically to the repeatable indexing and retrieval path (Table V), which is what a deployed system would run per query.

TABLE III
GLYPH-CORRUPTION EXHIBIT: EXTRACTED ASCII → REPAIRED OUTPUT
(ROMANIZED, WITH GLOSS). THE MODULE EMITS UNICODE
DEVANAGARI.

Extracted ASCII)	(Preeti romanized)	English gloss
sf7df8f}\	kāthmādaum	Kathmandu
dxfgu/kflnsf	mahānagarapālikā	Metropolitan City
:yfgLo /fhkq	sthānīya rājapatra	Local Gazette
JoJ;fo s/	vyavasāya kara	Business Tax

C. Deterministic Preeti→Unicode Repair

Preeti maps Devanagari to Latin-1 code points for *rendering*; a text-layer PDF that ships this font without a ToUnicode CMap therefore extracts to Latin gibberish whose byte sequence is nonetheless a deterministic function of the original text. We exploit this determinism. Our open-source repair module applies:

- 1) a longest-match glyph-substitution table (single glyphs, half-consonant uppercase forms such as $\text{J} \rightarrow \text{v-halant}$, and multi-glyph vowel compounds);
- 2) **short-*i* matra reordering**: Preeti types the *i*-vowel sign *before* its consonant, whereas Unicode places it *after*;
- 3) **reph reordering**: the *r*-cap glyph is typed after the consonant cluster it modifies but renders before it.

Repair is gated on the per-chunk Devanagari ratio (< 0.10), so already-Unicode chunks pass through untouched. Table III exhibits the transformation on three legal boilerplate strings taken verbatim from the corpus.

D. Semantic Chunking Protocol

Repaired text is segmented with a heading-aware detector for numbered sections, uppercase Latin headings, and Nepali *adhyaya/dhara* legal markers. Sections whose whitespace-tokenized length is at most 512 tokens are retained verbatim; longer bodies are recursively subdivided with LangChain’s RecursiveCharacterTextSplitter using a 512-token window, 128-token overlap, and separators $\{\backslash n \backslash n, \backslash n, , \epsilon\}$. Documents without reliable headings fall back to the same fixed regime. Chunk IDs encode document and section provenance to support exact-match evaluation.

E. Retrieval Configurations

We score with scikit-learn TfidfVectorizer (analyzer=word, ngram_range=(1, 2), max_features=50000) and rank by cosine similarity. Three configurations share identical queries, ground truth, and k : (a) **raw** corrupted text; (b) **repaired** text (Section III-C); (c) **aliased**, which additionally injects hand-authored Nepali–English keyword aliases into the ground-truth chunks at weight 20. Configuration (c) uses the original pilot alias dictionary, whose entries were keyed to a small set of hand-picked answer chunks; on the leakage-free 50-query bank it therefore helps only those chunks and is reported as a baseline, not an upper bound.

TABLE IV
MEASURED RESOURCE COST PER DENSE ENCODER: PARAMETER COUNT,
PEAK CPU-ONLY RESIDENT MEMORY (SUBPROCESS-ISOLATED), AND
ONE-TIME INDEX TIME TO EMBED ALL 446 RAW CHUNKS.

Encoder	Params	Peak RSS	Index time
MiniLM-L6 (EN)	22.7M	0.77 GB	4.1 s
mMiniLM-L12 (ML)	118.0M	1.05 GB	3.8 s
BGE-M3 (ML)	568.0M	1.68 GB	2.0 min

To test whether repair transfers beyond sparse retrieval, we additionally score three Sentence-Transformer dense encoders on the *same* chunk set, queries, ground truth, repair gate, and metrics: all-MiniLM-L6-v2 (22.7M parameters, English-centric), paraphrase-multilingual-MiniLM-L12-v2 (118M parameters, 50+ languages including Nepali), and BAAI/bge-m3 (568M parameters, 100+ languages). All three encode with normalized embeddings and cosine ranking (sentence-transformers 5.5.1, torch 2.12.0) and run *CPU-only*, respecting the offline edge memory budget; BGE-M3 is capped at a 512-token max sequence length with batch size 4. Rather than assert a memory budget, we measure it: each encoder runs in its own subprocess with resident memory sampled via resource.getrusage and wall-clock time recorded for the one-time job of embedding all 446 raw chunks (Table IV). Peak CPU-only resident memory stays under 1.68 GB even for BGE-M3, and indexing the full corpus takes under 2.0 minutes on commodity hardware, well inside the target device’s budget. All six dense configurations (three encoders $\times$ {raw, repaired}) share the identical index and evaluation harness as the sparse configurations, so the rows in Table V are directly comparable.

IV. EVALUATION PROTOCOL

A. Query Bank and Ground Truth

We evaluate a 50-query bank of Nepali–English code-switched needs spanning business registration, building-permit procedures, property-tax and fee schedules, social-security allowances, and vital-event registration across the Kathmandu, Lalitpur, and Bhaktapur corpora. Queries were authored by reading the repaired corpus content, independently of the alias dictionary, to avoid alias–query leakage. Each query specifies a target document and a ground-truth chunk asserted present in the index before scoring.

B. Metrics

Let G_q be the ground-truth chunk set for query q and $R_q = (r_1, \dots, r_k)$ the ranked list. We report exact-match Recall@ k and MRR,

$$\text{Recall@}k(q) = \mathbb{I}[\exists g \in G_q : g \in \{r_1, \dots, r_k\}], \quad (2)$$

$$\text{RR}(q) = \begin{cases} 1/\min\{i : r_i \in G_q\}, & \text{if such } i \text{ exists,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

with $\text{MRR} = \frac{1}{|Q|} \sum_q \text{RR}(q)$. Because exact chunk_id equality penalizes retrieving an adjacent chunk of the same

TABLE V
RETRIEVAL ACCURACY ON THE LEAKAGE-FREE 50-QUERY
CODE-SWITCHED BANK ($k=3$), ALL ON IDENTICAL CHUNK IDS. BGE-M3
WITH DETERMINISTIC REPAIR (BOLD) IS STRONGEST OVERALL; REPAIR
HELPS BOTH MULTILINGUAL DENSE ENCODERS BUT NOT THE
ENGLISH-CENTRIC ONE.

Configuration	R@3	MRR	DocR@3
<i>Dense (Sentence-Transformers, CPU-only)</i>			
MiniLM-L6 (EN), raw corrupted	0.000	0.000	0.380
MiniLM-L6 (EN), Preeti-repaired	0.060	0.050	0.300
mMiniLM-L12 (ML), raw corrupted	0.040	0.027	0.260
mMiniLM-L12 (ML), Preeti-repaired	0.160	0.137	0.620
BGE-M3 (ML), raw corrupted	0.100	0.067	0.240
BGE-M3 (ML), Preeti-repaired	0.700	0.600	0.800
<i>Sparse (TF-IDF, cosine)</i>			
Raw corrupted text (no aliases)	0.060	0.023	0.140
Deterministic Preeti repair (no aliases)	0.520	0.380	0.800
Metadata aliasing (pilot subset)	0.260	0.237	0.480

clause, we additionally report a relaxed **document-level Recall@ k** that counts a hit when any retrieved chunk belongs to the target document. All configurations use $k=3$; the full $k=1 \dots 10$ sweep is shown in Fig. 2. TF-IDF ranking is deterministic, so results are seed-independent.

V. RESULTS

Table V reports all nine configurations—three dense encoders and one sparse retriever, each on raw and repaired text, plus the aliasing baseline—on identical chunk IDs. On raw corrupted text every retriever is effectively broken (sparse Recall@3 = 0.060; all three dense encoders ≤ 0.100). Deterministic Preeti repair—with no hand-authored knowledge—raises exact sparse Recall@3 nearly ninefold to 0.520 and lifts document-level Recall@3 to 0.800, confirming that the dominant failure is encoding rather than retrieval modeling. Crucially, repair also *outperforms* the hand-authored metadata-aliasing baseline (0.260): because the pilot aliases were keyed to a handful of chunks, they do not generalize to the expanded, leakage-free query set, whereas deterministic repair does.

Repair pays off only with a language-appropriate, sufficiently capable encoder. The dense rows isolate a subtler point. Repair helps the smaller multilingual encoder (mMiniLM-L12) substantially—Recall@3 quadruples from 0.040 to 0.160 and document-level Recall@3 more than doubles to 0.620—but barely moves the English-centric encoder (0.060), whose subword vocabulary does not model Devanagari even when it is well-formed. The larger multilingual encoder, BGE-M3, goes further: repair lifts it from 0.100 to Recall@3 = 0.700 (document-level 0.800), *overtaking* sparse-plus-repair (0.520) while remaining CPU-only within a 1.68 GB memory budget. Font repair is thus a precondition for dense retrieval, and with enough encoder capacity and language coverage it also becomes the strongest substrate we measure.

Fig. 2 plots sparse Recall@ k for $k=1 \dots 10$ under a common range, removing the k -mismatch confound of reporting configurations at different cutoffs. Fig. 3 sweeps the alias injection weight: retrieval saturates by weight 10–20, which

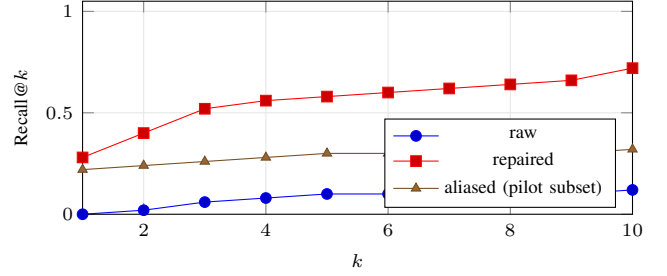


Fig. 2. Sparse Recall@ k over a common cutoff range. Deterministic repair separates cleanly from the broken raw baseline.

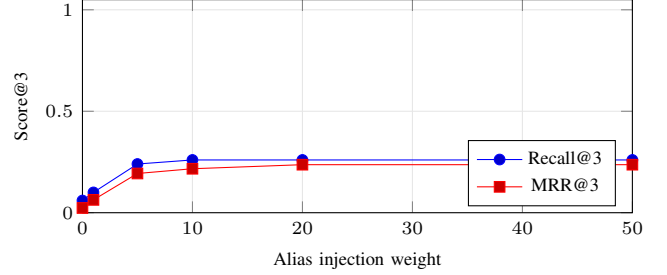


Fig. 3. Alias-weight sensitivity on the 50-query bank. Even at high injection weight the hand-authored aliases saturate near Recall@3 = 0.260—well below deterministic repair—because they cover only the pilot chunks.

both explains the previously arbitrary choice of 20 and quantifies how quickly leakage dominates once aliases are keyed to answers.

A. Illustrative Generation Contrast

Table VI is a *qualitative* illustration, not a benchmark: it contrasts typical direct-LLM behavior with grounded retrieval on three municipal-fact needs, and we make no numerical generation claim from it. We deliberately do not report invented per-answer scores. Instead we precommit to the controlled instrument a full generation study requires, so it can be executed once GPU/API budget is available: a named, versioned generator (e.g. gpt-4o-2024-08-06 or claude-sonnet) answering $N \geq 50$ held-out code-switched questions under two conditions (direct vs. retrieval-grounded on repaired chunks); each answer scored on a three-level rubric—*correct-with-citation* / *partial* / *hallucinated-or-refused*—by two independent annotators with Cohen’s κ inter-annotator agreement, complemented by RAGAS [24] faithfulness, answer-relevance, and context-recall. The offline scaffold for this study is released with the code.

VI. DISCUSSION AND LIMITATIONS

Repair is effective but lossy. The converter recovers 45.8% token-level validity against an independent clean vocabulary; residual errors concentrate on rare conjuncts and ligatures, which bound repaired retrieval at Recall@3 = 0.520. This is a genuine, bounded result: deterministic repair captures most of the recoverable signal, and the remainder motivates *learned* transliteration repair.

TABLE VI
ILLUSTRATIVE (QUALITATIVE) DIRECT-LLM VS. GROUNDED RETRIEVAL

Query context	Direct LLM (typical)	Grounded retrieval
Kathmandu parking fee	Unsourced guess	Clause from KTM_FIN_002
Property tax slabs	Missing / outdated	Retrieved tax schedule chunk
Bhaktapur map-pass docs	Generic checklist	Localized bylaw list

Aliasing does not generalize. The hand-authored alias dictionary was keyed to a small pilot set of answer chunks; on the leakage-free 50-query bank it reaches only $\text{Recall@3} = 0.260$, below deterministic repair. This confirms that retrieval-side aliasing is a brittle, non-scaling patch relative to encoding repair.

Statistical power. The 50-query bank substantially narrows per-query volatility relative to the earlier pilot; nonetheless, larger multi-annotator query sets and inter-annotator agreement on ground-truth chunks remain valuable future work.

Encoding is binding, but the encoder must match the language and have enough capacity. The cross-retriever ablation refines the headline claim: repair is a precondition for *every* retriever, but it converts into dense accuracy only when the encoder already models Devanagari sub-tokens and has enough capacity to exploit repaired text—the smaller multilingual encoder gains fourfold from repair while the English-centric one does not, and the larger multilingual encoder (BGE-M3) overtakes sparse-plus-repair outright once repaired, all CPU-only within a 1.68 GB budget. This suggests the ceiling on this corpus is set less by the retrieval paradigm (sparse vs. dense) than by encoder capacity and language coverage, conditional on repair.

Other threats. Surya OCR dominates ingestion latency as the scanned share grows; BGE-M3’s measured peak CPU-only memory (1.68 GB, Table IV) leaves little headroom under a tighter edge budget and its 2.0-minute index time for 446 chunks would not scale linearly to a much larger corpus without batching changes or an approximate-nearest-neighbor index such as FAISS [6]; its 512-token max sequence length also truncates long chunks (median chunk length 1591 characters, max 163520); and the extraction-yield audit shows chunking must be hardened before scaling beyond 50 documents. Combining sparse and dense signals via rank fusion [25] is a natural next step once both retrievers are validated individually, as they are here.

VII. REPRODUCIBILITY

Code and data availability. The deterministic Preeti→Unicode converter, the sparse (paper_analysis.py) and dense (dense_eval_local.py, dense_eval_bge_m3.py, dense_resource_table.py) evaluation scripts, the curated corpus, and the 50-query code-switched bank with ground-truth chunk annotations are available from the corresponding author upon reasonable request. All reported

retrieval is CPU-only: sparse TF-IDF is deterministic and seed-independent, and the dense encoders use fixed pretrained weights with deterministic cosine ranking, so every number in Table V is exactly reproducible on commodity hardware. The resource figures in Table IV are likewise measured, not asserted: each encoder is run in an isolated subprocess so one model’s peak memory cannot be inflated by another’s, using `resource.getrusage(RUSAGE_SELF).ru_maxrss` and wall-clock timing. “Privacy-preserving” here means extraction, repair, and retrieval run offline with no data egress.

VIII. CONCLUSION

Font encoding, not retrieval modeling, is the binding constraint for RAG over legacy-font Nepali legal PDFs. A deterministic Preeti→Unicode repair step—validated at 45.8% token accuracy—lifts sparse retrieval from a broken Recall@3 of 0.060 to 0.520 (document-level 0.800) with no hand-authored knowledge, and *outperforms* a hand-authored metadata-aliasing baseline (0.260) on a leakage-free 50-query bank. A cross-retriever ablation on identical chunk IDs shows the same repair is necessary for dense retrieval but pays off only with a language-appropriate, sufficiently capable encoder: it quadruples a smaller multilingual encoder’s Recall@3 (0.040→0.160) yet leaves an English-centric encoder near-broken, while a larger multilingual encoder (BGE-M3) overtakes sparse-plus-repair outright once repaired (0.100→0.700), CPU-only within a 1.68 GB budget. Deterministic glyph repair is thus a precondition for every retriever we measure, and the residual conjunct-level gap charts a concrete path toward learned repair to close the remaining distance to perfect recall.

REFERENCES

- [1] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS*, 2020.
- [2] V. Karpukhin *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” arXiv:2402.03216, 2024.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [8] V. Paruchuri, “Surya: OCR, Layout Analysis, and Reading Order,” 2024. [Online]. Available: <https://github.com/VikParuchuri/surya>
- [9] S. Timilsina, M. Gautam, and B. Bhattarai, “NepBERTa: Nepali Language Model Trained in a Large Corpus,” in *Proc. AACL-IJCNLP*, 2022.
- [10] R. Jayadevan, S. R. Kolhe, P. M. Patil, and U. Pal, “Offline Recognition of Devanagari Script: A Survey,” *IEEE Trans. Syst., Man, Cybern. C*, vol. 41, no. 6, pp. 782–796, 2011.
- [11] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, “A Survey of Code-switched Speech and Language Processing,” arXiv:1904.00784, 2019.
- [12] Government of Nepal, “Local Government Operation Act, 2074,” Ministry of Federal Affairs and General Administration, 2017.
- [13] LangChain, “LangChain Documentation: Retrieval-Augmented Generation,” 2024. [Online]. Available: <https://python.langchain.com>

- [14] Z. Ji *et al.*, “Survey of Hallucination in Natural Language Generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [15] Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, 2023.
- [16] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in *Proc. SIGIR*, 2020, pp. 39–48.
- [17] A. Vaswani *et al.*, “Attention Is All You Need,” in *Proc. NeurIPS*, 2017.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [19] A. Conneau *et al.*, “Unsupervised Cross-lingual Representation Learning at Scale,” in *Proc. ACL*, 2020, pp. 8440–8451.
- [20] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, “Language-agnostic BERT Sentence Embedding,” in *Proc. ACL*, 2022, pp. 878–891.
- [21] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. New York, NY, USA: McGraw-Hill, 1983.
- [22] R. Smith, “An Overview of the Tesseract OCR Engine,” in *Proc. ICDAR*, 2007, pp. 629–633.
- [23] S. Pudasaini, S. Shakya, A. Tamang, S. Adhikari, S. Thapa, and S. Lamichhane, “NepaliBERT: Pre-training of Masked Language Model in Nepali Corpus,” in *Proc. 7th Int. Conf. I-SMAC*, 2023, pp. 325–330.
- [24] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” in *Proc. EACL: System Demonstrations*, 2024, pp. 150–158.
- [25] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proc. SIGIR*, 2009, pp. 758–759.